

ENIDIA – TypeScript WebPage Rebuild

Project Plan

Client: ENIDIA – Tyler Cheng

Team Members: Maksym, Dylan, Joe

Date: 06/12/2025

1. Project Background & Rationale

ENIDIA currently operates a legacy marketing website (ENGAME) that was built using an older technology stack and design approach. The existing site:

- Has layout and scaling issues across devices and resolutions.
- Uses older React/TypeScript patterns that are harder to maintain.
- Lacks a modern, modular design system and clear component architecture.

To support ongoing marketing, partnerships, and growth, ENIDIA has requested a **modern rebuild** of the site, preserving the existing business functionality and content while upgrading the underlying technology and user experience.

This project will rebuild the website using **Next.js, TypeScript, TailwindCSS, and a modular component structure**, ensuring improved maintainability, responsiveness, and performance.

2. Project Goals & Objectives

Primary Goal:

Rebuild the ENIDIA marketing website into a modern, modular, responsive web application while preserving all critical content and behaviour from the legacy site.

Objectives:

1. Recreate all key public-facing pages from the legacy ENGAME website in a **Next.js + TypeScript** project.
 2. Implement a **modern responsive layout** that works consistently on desktop, tablet, and mobile devices.
 3. Develop a **reusable component library** (Navbar, Footer, Slideshow, Timeline, Trusted Companies, sections, buttons, forms, etc.).
 4. Match or improve the original **animations, slideshows, and timeline interactions** using Framer Motion or equivalent.
 5. Improve **performance, accessibility, and SEO**, aiming for **Lighthouse scores above 90** in core categories.
 6. Deploy the new site to a live environment (e.g. **Vercel**) and provide documentation for ongoing maintenance.
-

3. Project Scope

3.1 In Scope

- Rebuild of all public-facing pages from the legacy site, including (but not limited to):
 - Home
 - Our Story
 - Brand Owner
 - Gamer
 - FAQ
 - Free Trial / Pricing
 - Blog listing and blog detail pages (if present)
- Implementation of a **modern file/folder structure** for the codebase (e.g. app/, components/, lib/, public/assets/).
- Creation of a **design system** using reusable Tailwind-based components.
- Recreation of **interactive features**, such as:
 - Strategic Plan slideshow (2021 / 2022 / 2023)
 - Timeline components
 - “Trusted by the world’s leading companies” logo strip with hover effects
- Front-end integration with the **existing backend** (test environment provided by client).
- Basic **analytics hooks** (where appropriate) and support for future AI-based recommendations (optional, if time allows).
- **Documentation:** project setup instructions, component usage notes, and maintenance guidance.

3.2 Out of Scope

- Building or rebuilding the backend services or databases.
- A full CMS or admin dashboard for content editing.
- Payment integration, account billing, or subscription management.
- Complex analytics dashboards or production-grade AI recommendation systems (beyond basic hooks or placeholders).
- Long-term hosting management and monitoring beyond initial deployment.

4. Project Deliverables

1. **Modern ENIDIA Marketing Website**

- Next.js + TypeScript project, deployed and functional.
 - 2. **Reusable Component Library**
 - Components for layout, navigation, buttons, hero sections, cards, timeline, slideshow, and partners strip.
 - 3. **Responsive, Accessible UI**
 - Mobile-first design, tested on multiple viewports.
 - Accessibility improvements aligned with **WCAG 2.1 AA** where feasible.
 - 4. **Performance & Accessibility Report**
 - Lighthouse scores and notes on performance, accessibility, and SEO improvements.
 - 5. **Technical Documentation**
 - How to run, build, and deploy the project.
 - File structure description.
 - Notes on integrating with the existing backend.
 - 6. **Project Artefacts for Assessment**
 - Project Plan, Sprint Plans, Risk Assessment, Meeting Minutes, and final Reflection documents as required by the module.
-

5. Project Assumptions

- The client (ENIDIA, via Tyler Cheng) will supply or approve all brand assets (logos, images, colour palette, copy).
 - The existing backend/test environment will be available and accessible for integration and testing.
 - Team members (Maksym, Dylan, Joe) will have regular access to required tools (VS Code, GitHub, Node.js, browsers).
 - The scope will remain focused on rebuilding the existing site, not a complete rebrand.
-

6. Project Constraints

- **Time constraints:** The project must be completed within the academic semester and within the agreed sprint schedule (e.g. by the end of Sprint 3 of this term).
- **Team availability:** Student timetables, deadlines for other modules, and limited overlapping working hours.
- **Technical constraints:** The rebuilt site must work with the **pre-existing backend**, limiting certain tech choices.

- **Assessment requirements:** Documentation and deliverables must align with university marking criteria (e.g. Agile usage, evidence of sprints, risk logs).
-

7. Stakeholders

Stakeholder	Role / Interest
ENIDIA – Tyler Cheng	Client; wants a modern, reliable marketing site matching or improving the legacy design.
Maksym	Developer (front-end + integration); responsible for structure, some components, and documentation.
Dylan	Developer (front-end); responsible for key pages and UI behaviour.
Joe	Developer (support / QA); may assist with testing and integration.
Module Tutor / Supervisor	Oversees academic quality and confirms that assessment requirements are met.

8. Work Breakdown Structure (WBS)

1. Project Initiation

- 1.1 Confirm client requirements and constraints.
- 1.2 Review legacy site and source code.
- 1.3 Identify pages, assets, and functionality to preserve.

2. Planning

- 2.1 Draft Project Plan.
- 2.2 Create Sprint Plan (sprints and goals).
- 2.3 Define Trello board workflow and roles.

3. Design & Architecture

- 3.1 Define folder structure (app/, components/, lib/, public/assets/).
- 3.2 Choose tech stack details (Next.js version, Tailwind, Framer Motion).
- 3.3 Map legacy components to new structure (component mapping document).

4. Component Development

- 4.1 Global layout (AppShell, Navbar, Footer).
- 4.2 UI elements (Button, Input, Modals, Sections).
- 4.3 Marketing components (Timeline, Slideshow, TrustedCompanies, Hero sections).

5. Page Development

- 5.1 Home page.
- 5.2 Our Story.
- 5.3 Brand Owner & Gamer pages.
- 5.4 FAQ, Pricing, Free Trial.
- 5.5 Blog listing + detail pages (if included in scope).

6. Testing & QA

- 6.1 Functional testing of all pages.
- 6.2 Responsive testing on multiple devices.
- 6.3 Accessibility checks.
- 6.4 Performance tests (Lighthouse).

7. Deployment & Handover

- 7.1 Deploy to Vercel (or alternative).
- 7.2 Final client review and tweaks.
- 7.3 Documentation and code handover.
- 7.4 Final academic submission.

9. Timeline / High-Level Schedule

You can adapt this to your actual sprint dates.

Week / Sprint Activities

Week 1 (Sprint 1) Setup GitHub repo, Next.js project, Tailwind, import assets, map legacy → new pages, create Project Plan.

Week 2 (Sprint 1–2) Implement layout (Navbar, Footer, AppShell), base components, start Our Story page structure.

Week 3 (Sprint 2) Complete Our Story (timeline, strategic plan slideshow), Trusted Companies, fix image structure.

Week 4 (Sprint 2–3) Implement remaining pages (Home, Brand Owner, Gamer, FAQ, Pricing).

Week 5 (Sprint 3) Testing, bug fixing, performance and accessibility improvements, cross-browser checks.

Week 6 (Sprint 3) Final polish, deployment to HostServer, complete documentation, prepare Interim Presentation / Final Report.

10. Resource Requirements

Team:

- Maksym – front-end, architecture, documentation
- Dylan – front-end pages, animations, layout fixes
- Joe – QA, integration assistance, test support

Tools & Technology:

- Node.js + npm
 - Next.js (App Router)
 - React 18 + TypeScript
 - TailwindCSS
 - Framer Motion (for animations)
 - Git & GitHub Classroom repo
 - VS Code (with ESLint, Prettier, Copilot optional)
-

11. Communication Plan

Audience	Channel	Frequency
Internal Team	Discord / WhatsApp / Teams	Frequent (as needed, several times per week)
Client (Tyler)	Email or online meeting	Approximately once per week or per major milestone
Tutor / Supervisor	Lecture / Lab sessions / Email	As required (checkpoints, sprint reviews)
Documentation	GitHub (README, docs)	Continuous updates

12. Risk Management

ID	Risk Description	Impact	Likelihood	Mitigation
R1	Time overruns due to complexity	High	Medium	Use sprints with clear goals; regularly review Trello board; prioritize core pages first.

ID	Risk Description	Impact	Likelihood	Mitigation
R2	Issues integrating with pre-existing backend	Medium	Medium	Clarify API requirements early; use a test environment; mock APIs where needed.
R3	Inconsistent UI due to multiple devs	Medium	Medium	Agree on component library & Tailwind utility conventions; use shared components.
R4	Performance issues from large assets	Medium	Low–Medium	Optimise images (Next/Image), lazy loading, compress assets.
R5	Git conflicts / lost work	Medium	Medium	Use feature branches; pull frequently; commit often with good messages.

13. Quality Plan

Quality will be ensured through:

- **Code quality:**
 - Use of TypeScript types and linting.
 - Clear component boundaries and folder naming conventions.
- **UI / UX quality:**
 - Visual comparison with legacy site (pixel-approximate matching).
 - Consistent typography, spacing, and colour usage via Tailwind.
- **Performance & accessibility:**
 - Lighthouse scores (target: > 90 for Performance, Accessibility, Best Practices, SEO).
 - Basic keyboard navigation checks and alt-text on images.
- **Testing:**
 - Manual testing on multiple devices and browsers.
 - Use of test URLs for client to validate behaviour.

14. Change Control Process

1. Any significant change (scope, new feature, design change) is logged in **Trello** or GitHub Issues.
2. The team reviews the impact (time, complexity, risk).
3. If accepted, the task is added to a future sprint.
4. The client (Tyler) is informed of major changes that affect delivery or behaviour.

15. Testing Strategy

- **Functional Testing:**
Ensure each page works as expected (navigation, buttons, links, forms, slideshows).
- **Responsive Testing:**
Test at common breakpoints: mobile (e.g. 375px), tablet (768px), desktop (1024px+).
- **Cross-Browser Testing:**
Chrome, Firefox, Edge, and Safari where possible.
- **Performance Testing:**
Run Lighthouse and optimise images, bundling, and layout where necessary.
- **Regression Checks:**
Compare against legacy site to ensure no core functionality is lost.

16. Handover Plan

At the end of the project:

- GitHub repository is finalised and cleaned (no secrets, clear README).
- Documentation (setup, build, deploy, structure) is completed and stored in /docs or /Design Documents.
- A short walk-through (live or recorded) is done to explain components and deployment.
- Access details are handed over.

17. Acceptance Criteria

The project will be accepted when:

- All key pages from the legacy site have been rebuilt and are functional.
- Visual design closely matches the original, with acceptable modern improvements.
- Site is fully responsive and passes basic accessibility checks.
- Lighthouse scores meet agreed targets (e.g. > 90 where feasible).
- The site is deployed successfully, and the client can access and test it.
- Project documentation is complete and understandable.

18. Budget Summary

This is a **student project**, so no commercial budget is allocated.

Item	Cost
Hosting	£0
Development tools (VS Code, GitHub, Node)	£0
Assets (logo, images provided by client)	£0
Total Project Cost	£0

19. Success Metrics

- Legacy site fully replicated in a modern codebase.
- Lighthouse Performance / Accessibility / SEO scores > 90.
- Positive feedback from client (Tyler) on usability and look & feel.
- Clean, well-structured GitHub repo suitable for review and marking.
- Team members understand and can extend the codebase after handover.

20. Project Closure

The project is considered closed when:

1. All acceptance criteria are met.
2. The site is deployed and tested by the client.
3. All documentation and design artefacts are delivered.
4. Final meeting or feedback is recorded.
5. Academic submission is completed and confirmed.

Appendix A — Approval & Sign-Off Page

This page confirms that the stakeholders acknowledge the contents of the Project Plan and accept their roles in delivering the ENIDIA – TypeScript WebPage Rebuild Project.

Client Approval

I hereby approve the project definition, deliverables, scope, risks, and timeline as outlined in this Project Plan.

Client Name: *Tyler Cheng*

Signature: _____

Date: _____

Project Team Signatures

Team Member	Signature	Date
Maksym	_____	_____
Dylan	_____	_____
Joe	_____	_____

Supervisor Acknowledgement

Name: Ji-Jian Chin

Signature: _____

Date: _____